Numpy

- *Numpy arrays*
  - Useful for most numerical calculations and plotting

```
import numpy as np
```

```
A=[1,2,3]; A=np.array(A) # same as A=np.array([1,2,3]), A is a 1D array
```

```
print(A.shape)
```

```
(3,)
```

```
element_first=A[0]; element_last=A[-1]; print(element_first, element_last)
```

```
2 1
```

```
A=np.array([[1,2,3],[4,5,6]]); print(A); print(A.shape) # A is a 2D array; 2x3 matrix
```

```
[[1 2 3]
 [4 5 6]]
(2, 3)
```

```
row_0=A[0,:]; print(row_0)
```

```
[1 2 3]
```

```
col_1=A[:,1]; print(col_1)
```

```
[2 5]
```

```
Nrow=A.shape[0]; Ncol=A.shape[1]; print(Nrow, Ncol)
```

```
2 3
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *More on array indices*
  - Try the following cell-by-cell

```
A=np.array([2,3,1,3,1]); print(A)
```

```
print(A[:])      # all elements (full slice; start and stop are default)
```

```
print(A[::+1])   # all elements with explicit step = +1 (same as A[:])
```

```
print(A[::-1])   # all elements in reverse order (step = -1)
```

```
A_sorted_asc=np.sort(A); print(A_sorted_asc)
```

```
A_sorted_desc=A_sorted_asc[::-1]; print(A_sorted_desc) # [::-1] means all row, all col
```

```
index_A_asc=np.argsort(A); print(index_A_asc)
```

```
print('Minimum value of A is:',A_sorted_asc[0])

index_small=index_A_asc[0]
print('Minimum value of A is:',A[index_small],' with index ',index_small)
```

```
print('Maximum value of A is:',A_sorted_asc[-1])
print('Maximum value of A is:',A_sorted_desc[0])

index_big=index_A_asc[-1]
print('Maximum value of A is:',A[index_big],' with index ',index_big)
```

```
index_A_desc=index_A_asc[::-1]; print(index_A_desc)
```

```
index_big=index_A_desc[0]
print('Maximum value of A is:',A[index_big],' with index ',index_big)
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *Creating (or initializing) numpy arrays*
  - Try the following cell-by-cell

```
x = np.zeros(3); print(x)
```

```
x = np.ones(3); print(x)
```

```
x = np.zeros((4,3)); print(x)
```

```
x = np.zeros(3, dtype='int'); print(x)
```

```
x = np.zeros(3, dtype='float'); print(x)
```

```
# create an array of given size with evenly spaced values
xi, xf, N_x = 0, 1, 10 # initial, final, number
x = np.linspace(xi, xf, N_x); print(x)
```

```
# creates an array with evenly spaced values of given step size
xi, xf, dx = 0, 1, 0.1 # initial, final, step
x = np.arange(xi, xf, dx); print(x)
```

```
# array with uniformly sampled random numbers
x_rand1 = np.random.rand(2); print(x_rand1,'\n','-'*20)
x_rand2 = np.random.rand(2,2); print(x_rand2,'\n','-'*20)
x_rand2 = np.random.rand(2,2,2); print(x_rand1,'\n','-'*20)
```

```
np.random.choice(x_rand1) # choice method gets one random value from a 1D array
```

```
x = np.zeros(3, dtype='int16'); print(x) # 16-bit (2-byte) per element array with zeros
```

```
print(np.iinfo(np.int16).min, np.iinfo(np.int16).max) # -2**15, 2**15-1
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

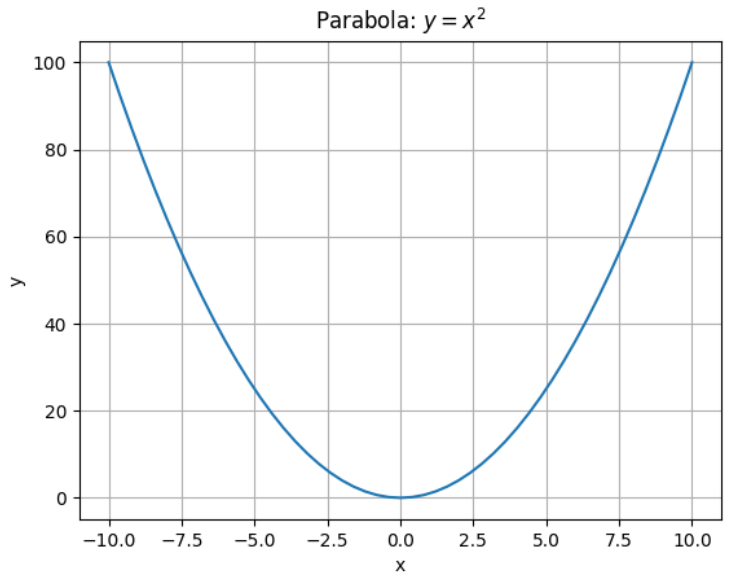
Basic plotting with **matplotlib.pyplot**

```
import numpy as np
import matplotlib.pyplot as plt

xmin = -10; xmax = 10; Ngrids = 51

x = np.linspace(xmin, xmax, Ngrids)
y = x**2

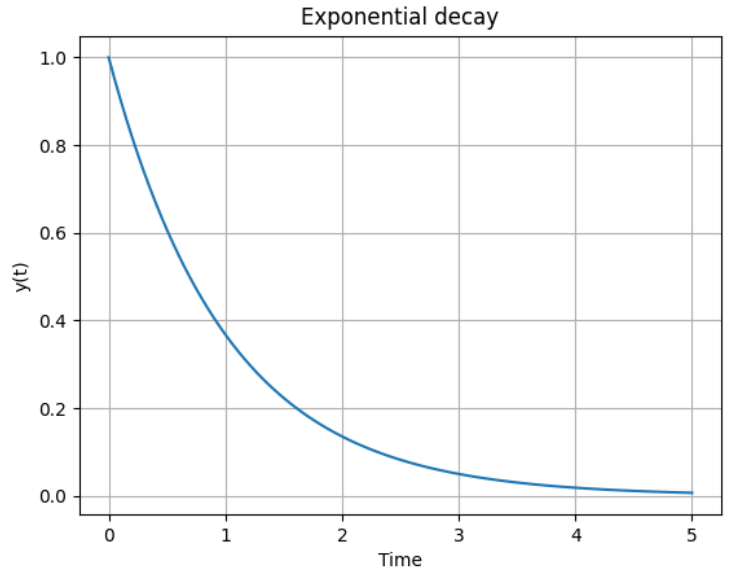
plt.plot(x, y)
plt.title("Parabola: $y = x^2$")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.savefig('parabola.png') # png, pdf, or svg
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 5, 100)
y = np.exp(-t)

plt.plot(t, y)
plt.title("Exponential decay")
plt.xlabel("Time")
plt.ylabel("y(t)")
plt.grid()
plt.show()
```



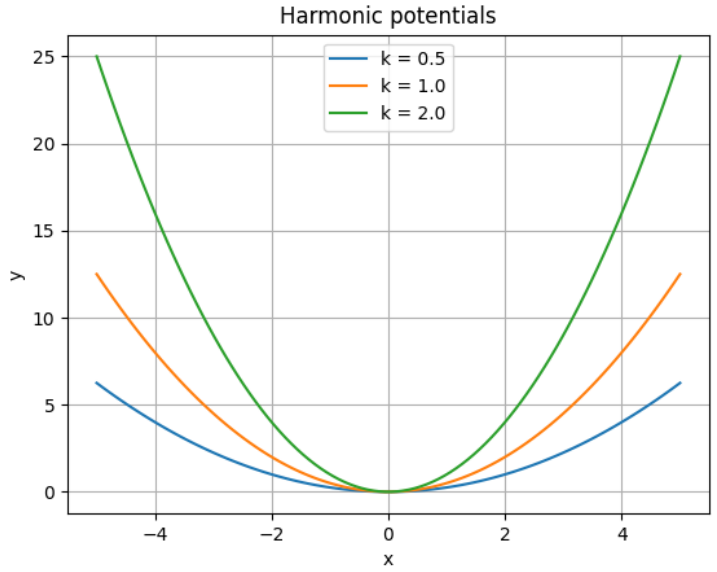
© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)

k=0.5; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=1.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=2.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")

plt.title("Harmonic potentials")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```

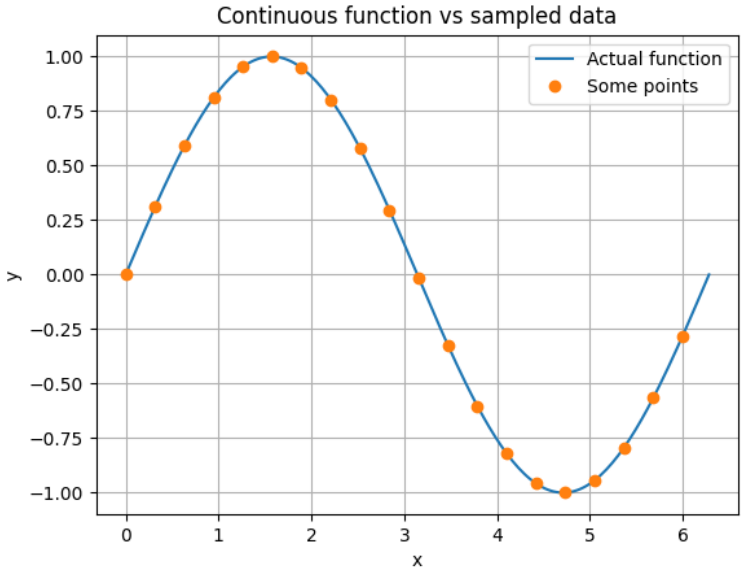



```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 200); y = np.sin(x)
a=x[::10]; b= y[::10] # a=x[start : stop : step], pick every 10th po

plt.plot(x, y, label="Actual function")
plt.plot(a, b, 'o', label="Some points")

plt.title("Continuous function vs sampled data")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

parametric plot

```
import numpy as np
import matplotlib.pyplot as plt

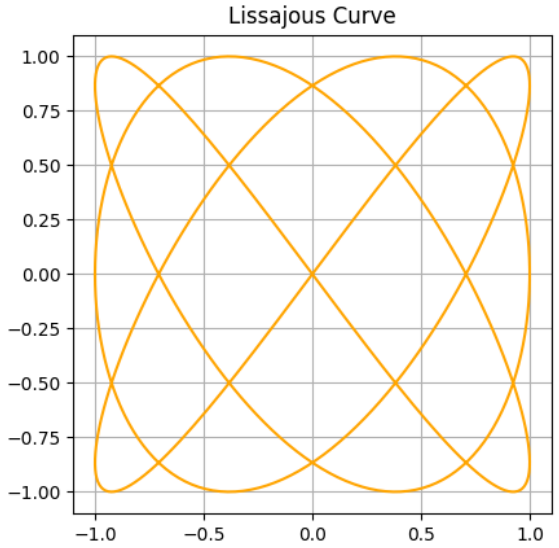
t = np.linspace(-np.pi, np.pi, 500)

A = 1; B = 1; a = 3; b = 4

x = A*np.sin(a*t + np.pi/2)
y = B*np.sin(b*t)

fig, ax = plt.subplots()  # create figure and axes
ax.plot(x, y, color="orange")
ax.set_aspect("equal")   # equal scaling: 1 unit in x = 1 unit in y

plt.title("Lissajous Curve")
plt.grid()
plt.show()
```



contour plot

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

def f(x, y):
    return x**2 + y**2

x = np.arange(-2.0, 2.0, 0.1)
y = np.arange(-2.0, 2.0, 0.1)

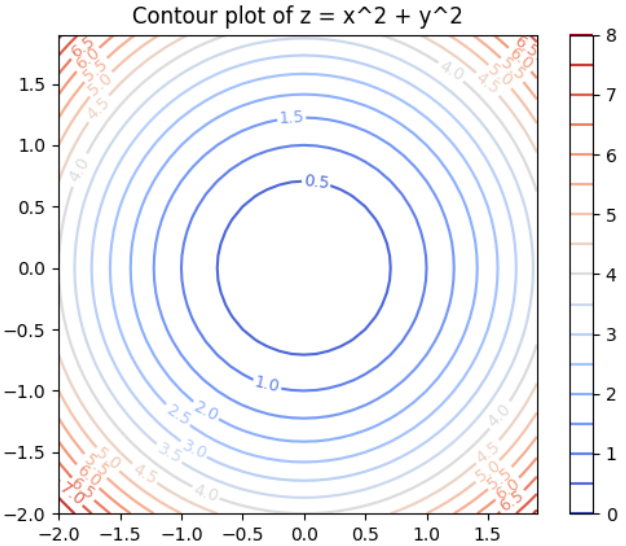
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

levels = np.arange(0, 8.5, 0.5)

fig = plt.figure()
ax = fig.add_subplot(111)

CS = ax.contour(X, Y, Z, levels, cmap=mpl.cm.coolwarm)
ax.set_aspect("equal", adjustable="box")

plt.clabel(CS, inline=True, fontsize=9)
plt.colorbar(CS)
plt.title("Contour plot of z = x^2 + y^2")
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA